

Vorlesung Cybersicherheit, Sommersemester 2025

# Digitale Signaturen, Hash Funktionen, Message Authentication Codes

Prof. Dr.-Ing. Lucas Vincenzo Davi

Fakultät für Informatik

Arbeitsgruppe Systemsicherheit <https://www.syssec.wiwi.uni-due.de/>

Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi

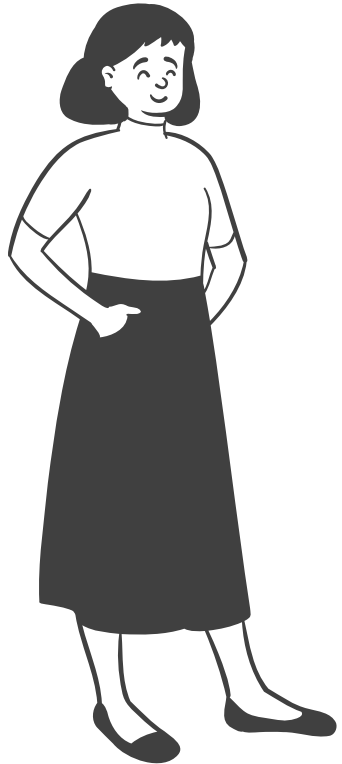
- Rückblick
  - Das RSA Kryptosystem
    - Schlüsselgenerierung
    - Verschlüsselung und Entschlüsselung
  - RSA-Beschleunigung mittels Square-and-Multiply
- Themen heute
  - Digitale Signaturen
  - Hash Funktionen
  - Message Authentication Code (MAC)

# Digitale Signaturen

- Elektronisches Unterschreiben von digitalen Dokumenten
- Eine signierte Nachricht kann eindeutig dem Sender zugeordnet werden (**Nichtzurückweisbarkeit** und **Authentisierung**)
- Zur Erinnerung:
  - Symmetrischer Kryptografie kann nicht genutzt werden, um Nichtzurückweisbarkeit zu gewährleisten
- Digitale Signaturen gewährleisten auch Nachrichten**integrität**
- Für digitale Signaturen nutzen wir asymmetrische Kryptografie

# Prinzip der Digitalen Signaturen

Alice



$k_{pub}$



$(x, s)$



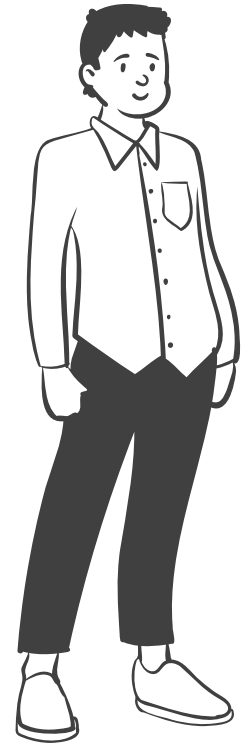
$Verify_{k_{pub}}(s, x) = true/false$

$k = (k_{pub}, k_{pr})$

Select  $x$

$s = sig_{k_{pr}}(x)$

Bob





## Beachte:

- Digitale Signaturen können mit allen bekannten asymmetrischen Kryptografie Verfahren umgesetzt werden, d.h. RSA, Diffie-Hellman und Elliptischen Kurven
- Wir besprechen in dieser Vorlesung das RSA Signaturverfahren

## Ablauf:

- Schlüsselerzeugung wie bei RSA-Verschlüsselung:
  - Öffentlicher Schlüssel:  $(n, e) = k_{pub}$
  - Privater Schlüssel:  $(d) = k_{pr}$
- Die Signatur wird mittels des privaten Schlüssels über den Klartext erzeugt
- Die Verifikation der Signatur wird mittels des öffentlichen Schlüssels durchgeführt

# Beispiel für RSA-basierte Digitale Signaturen

Alice

←  $(n, e)$

$$x' = s^e \bmod n$$
$$= 16^3 \bmod 33$$

$$x' = 4 \bmod 33$$

$$x' = x ? \Rightarrow 4 = 4 \checkmark$$

Bob

$$\mathcal{E}_{pr} = (d), \mathcal{E}_{pub} = (n, e)$$

$$n = 33, e = 3$$

$$d = 7, x = 4$$

$$s = x^d \bmod n$$
$$= 4^7 \bmod 33$$
$$= \underline{16} \bmod 33$$

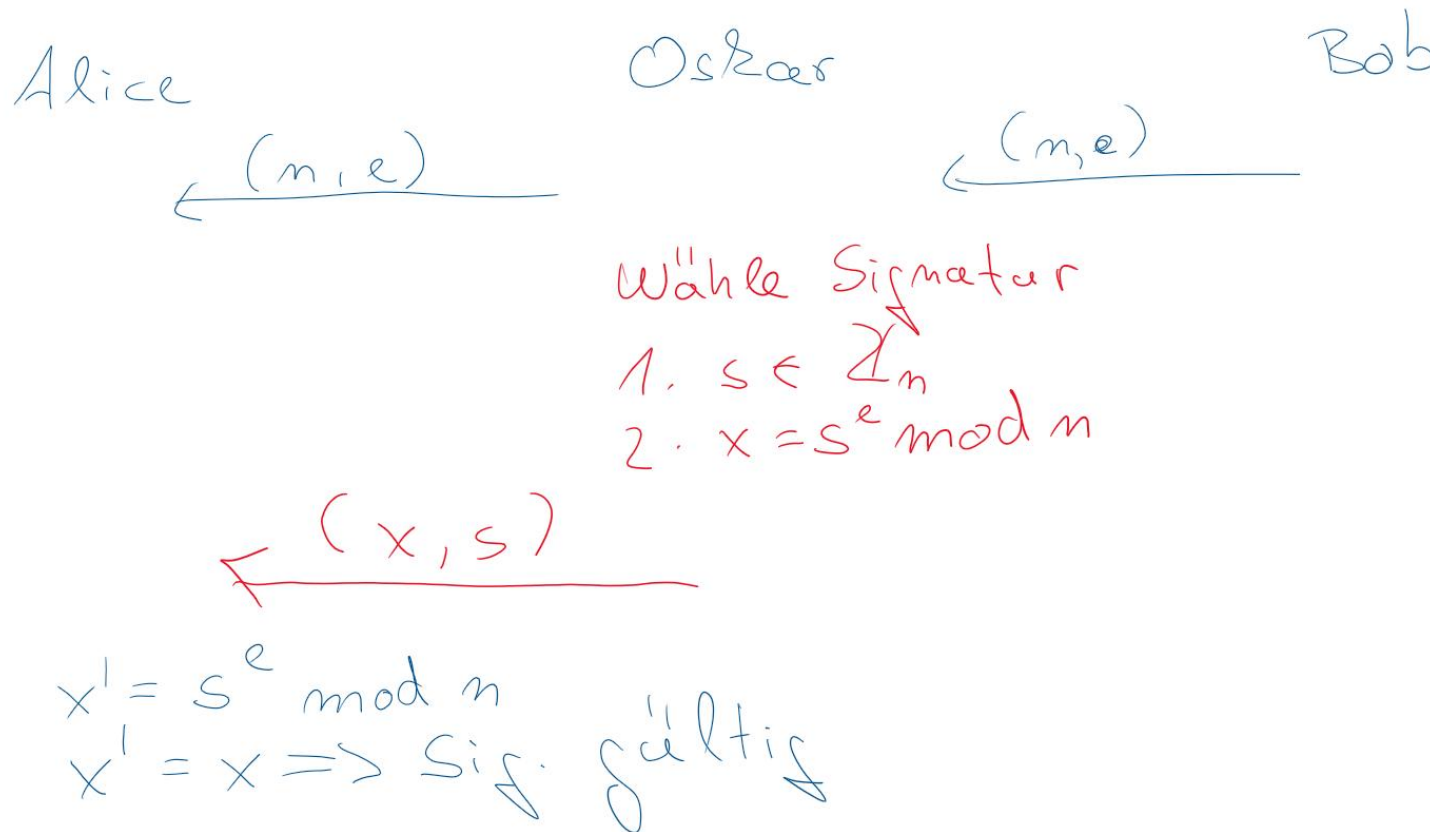
←  $(4, 16)$

# Man-in-the-Middle (MITM) Angriff

- Bis jetzt: Passiver Angreifer Oskar
  - Abhören des Kanals, um Vertraulichkeit zu verletzen
- Im Folgenden betrachten wir einen aktiven Angreifer
  - Dieser Angreifer kann Nachrichten abfangen und verändern
- Beim Man-in-the-Middle (MITM) Angriff tauscht der Angreifer öffentliche Schlüssel aus
- Für den Angriff nehmen wir an, dass Bob neben seinen öffentlichen Schlüssel auch seine ID über den Kanal schickt

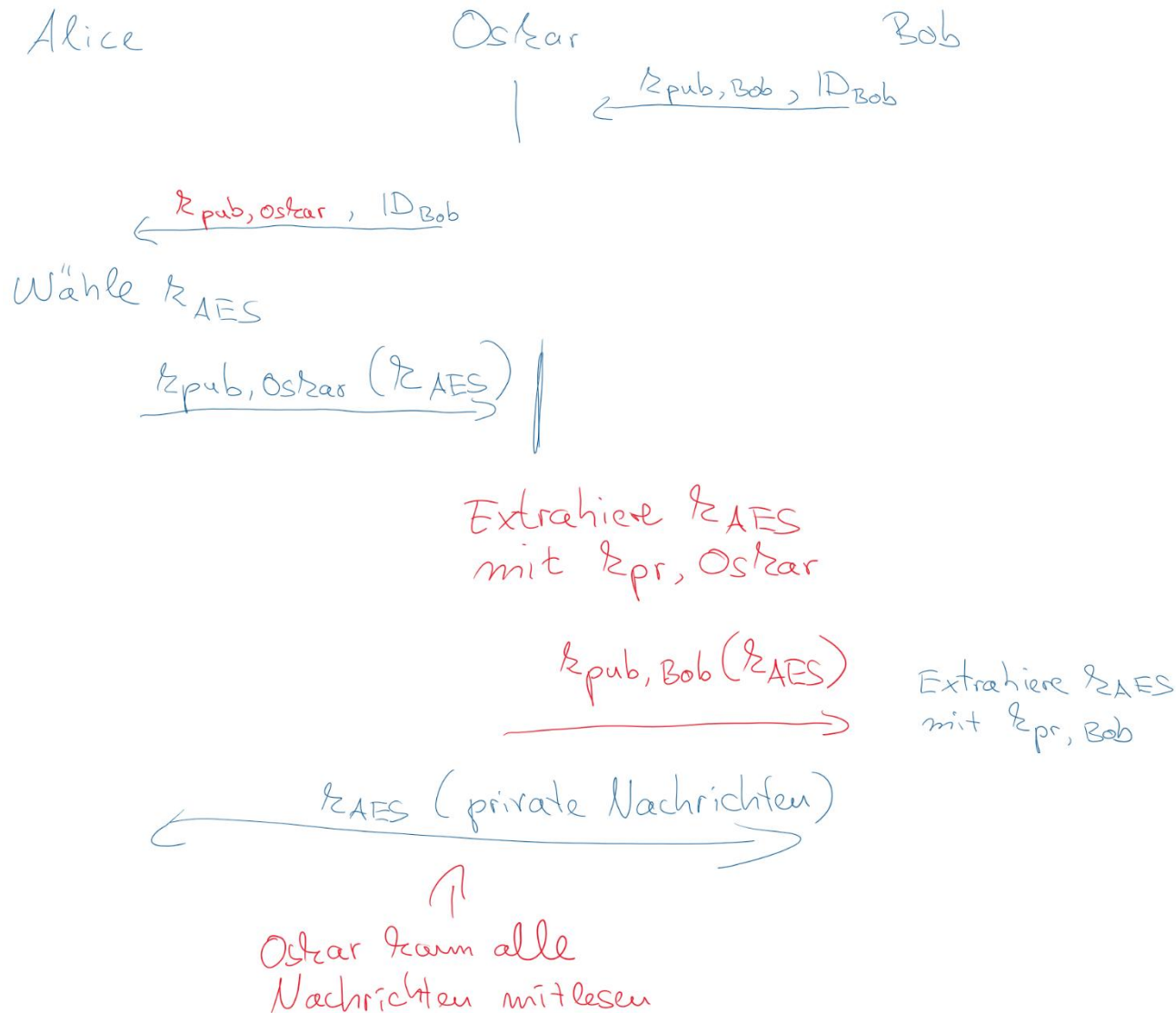


# Denial-of-Service Angriff auf RSA-Signatur



- RSA-Signaturen in der Praxis verhindern diesen Angriff über Padding
  - Nachrichten (das x in der Gleichung) müssen ein bestimmtes Format haben, z.B. 128 Nullen am Ende der Nachricht

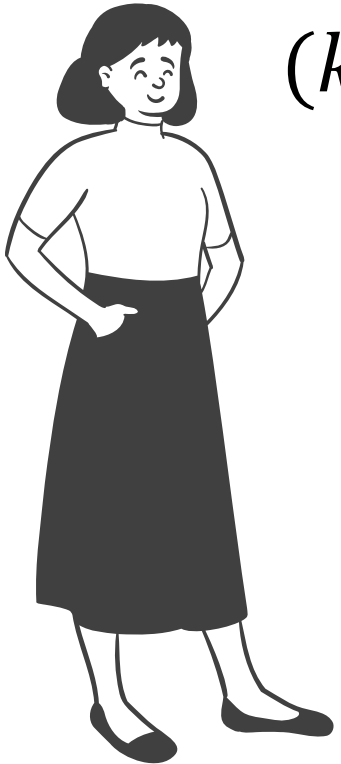
# Man-in-the-Middle (MITM) auf Public Key Austausch



- Grundproblem:
  - Wir brauchen für asymmetrische Kryptografie zwar keinen geheimen aber einen authentisierten Kanal, um öffentliche Schlüssel zu verteilen
- Lösung:
  - Einsatz von Zertifikaten
  - Aufbau:
    - $Cert_A = [ (k_{pubA}, ID_A), sig_{k_{prCA}}(k_{pubA}, ID_A) ]$
  - Zertifikate werden von Zertifizierungsstellen (Certificate Authorities, CAs) ausgestellt
  - Die öffentlichen Schlüssel zur Zertifikatsprüfung sind bereits im Browser vorinstalliert

# Protokoll für das Ausstellen von Zertifikaten

Alice

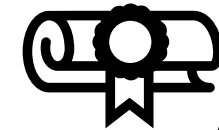


$(k_{pub,A}, k_{pr,A})$

$RQST(k_{pub,A}, ID_A)$



$(k_{pub,CA}, k_{pr,CA})$



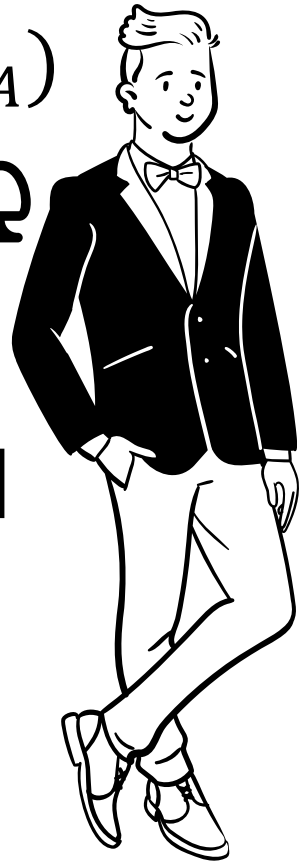
$Verify ID_A$

$cert_A = [ (k_{pubA}, ID_A), sig_{k_{prCA}}(k_{pubA}, ID_A) ]$

$cert_A$



Certificate Authority



# Zertifikate in der Praxis

- Public-Key-Infrastruktur (PKI)
  - Eine CA mit Unterstützungsmechanismen zum Betreiben einer CA (Identifizierung der Benutzer, Rückruf, sicheres Übertragen des öffentl. Schlüssel der CA)
- Rechts ein Beispiel für den Aufbau eines X.509 Zertifikat

|                                                                                                       |
|-------------------------------------------------------------------------------------------------------|
| Seriennummer                                                                                          |
| Schlüsselinformationen<br>asymm. Algorithmus<br>Parameter                                             |
| Aussteller                                                                                            |
| Gültigkeit<br>von<br>bis                                                                              |
| Zertifikatinhaber                                                                                     |
| Schlüsselinformationen Zertifikatinhaber<br>asymm. Algorithmus<br>Parameter<br>öffentlicher Schlüssel |
| Signatur                                                                                              |

# Hash Funktionen



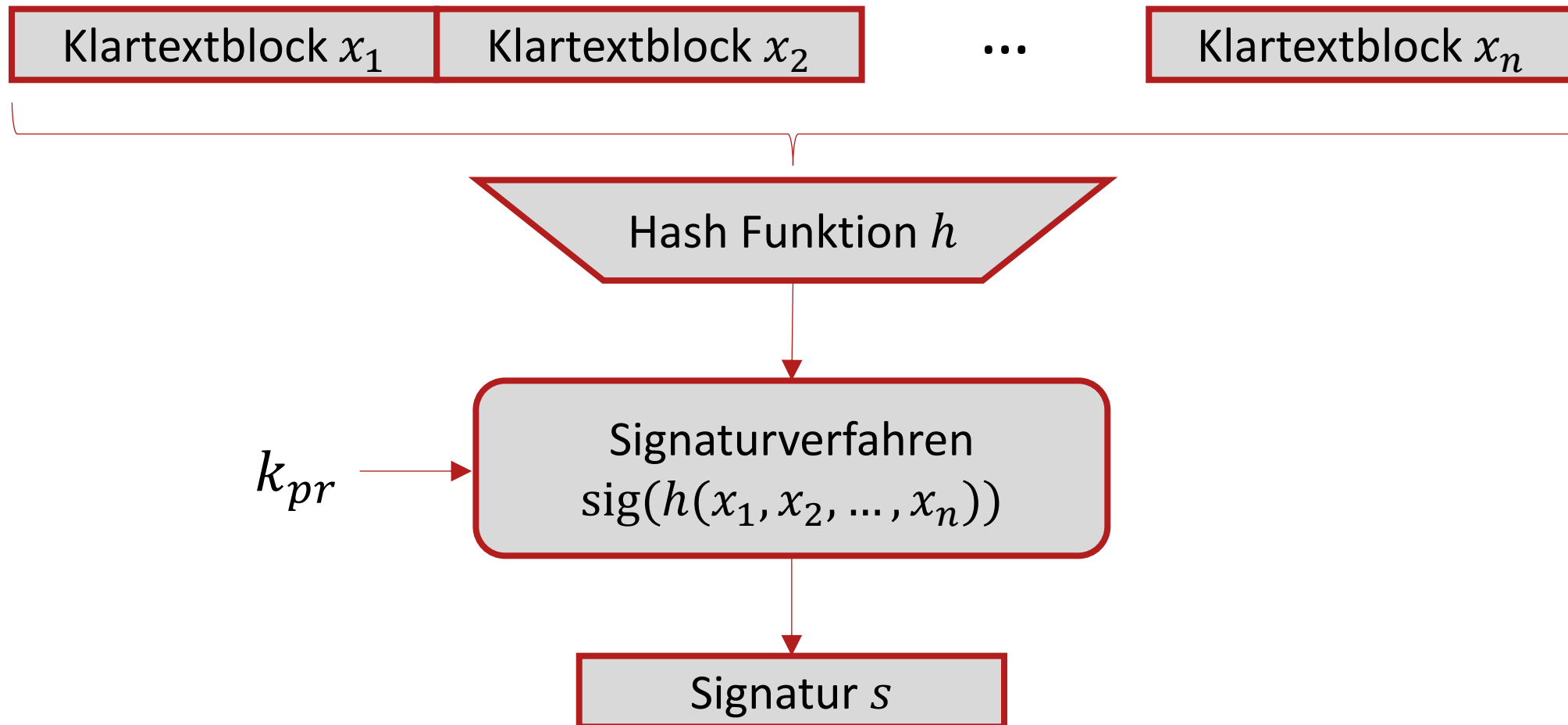
Beachte:

- Hash Funktionen haben keinen Schlüssel, haben aber viele Anwendungen in der Kryptografie

## Hash Funktionen und digitale Signaturen

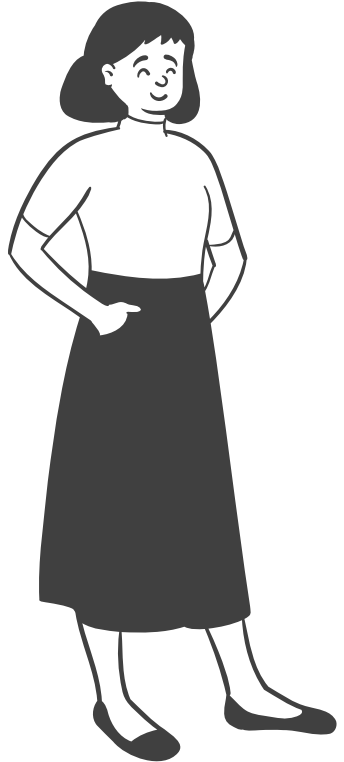
- Bei Signaturverfahren ist die Signatur auf die Nachrichtenlänge begrenzt
- Große Nachrichten (z.B. größer als 2048 Bits bei RSA-2048) können nicht einfach signiert werden
  - Betriebsmodi wie bei Blockchiffren sind eher nicht geeignet, da der Rechenaufwand für jeden Block hoch ist
- Hash Funktionen können hier Abhilfe schaffen

# Prinzip von Hash Funktionen bei Digitalen Signaturen



# Protokoll für Digitale Signaturen mit Hash-Funktionen

Alice



$k_{pub}$



$(x, s)$



$$z' = h(x)$$

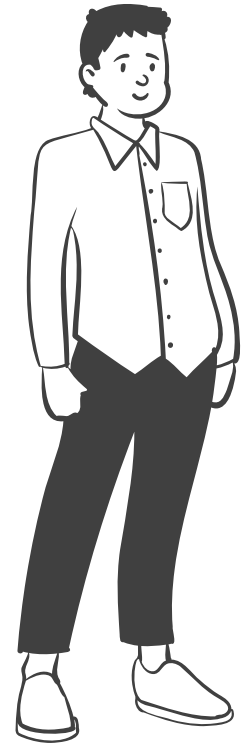
$$\text{verify}_{k_{pub}}(s, z') = \text{true/false}$$

$$k = (k_{pub}, k_{pr})$$

$$z = h(x)$$

$$s = \text{sig}_{k_{pr}}(z)$$

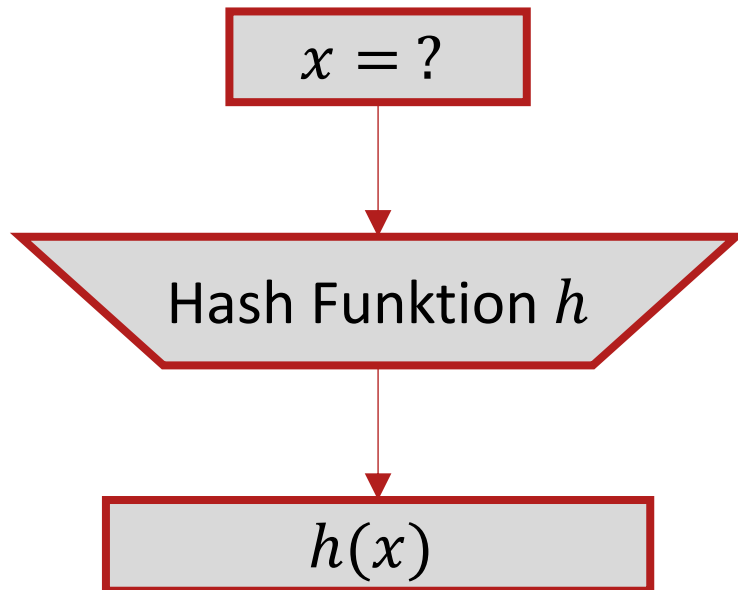
Bob



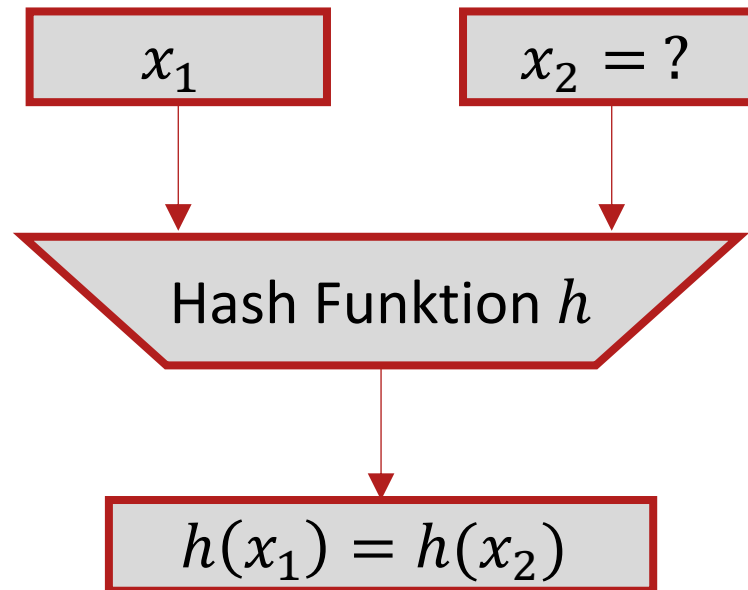
- Hohe Effizienz selbst bei langen Nachrichten
- Hash Funktionen erzeugen einen Fingerabdruck (fingerprint) fixer Länger aus einem beliebig langen Input
- Die Länge des Fingerabdrucks ist häufig 128 - 512 Bit
- Selbst kleinste Änderungen an der Nachricht führen zu großen Änderungen im Hash Resultat
  - Demo für SHA-256: <https://andersbrownworth.com/blockchain/hash>

# Sicherheitseigenschaften von Hash Funktionen

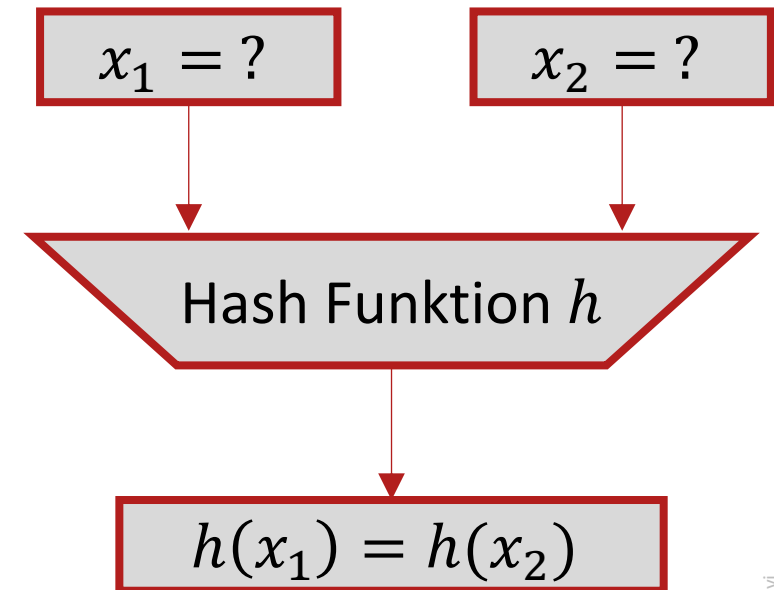
Urbildresistenz



Schwache Kollisionsresistenz



Starke Kollisionsresistenz



- Ziel:
  - Aus einem gegebenen Hash-Wert soll es nicht möglich sein den Eingangswert zu berechnen
- Beispiel:
  - Symmetrisch verschlüsselte Nachricht mit RSA-Signatur:  
 $(e_k(x), sig_{k_{pr}}(z))$  mit  $z = h(x)$
  - Falls Urbildresistenz nicht gegeben ist, kann Oskar die Nachricht entschlüsseln, denn dann wäre es möglich  $h^{-1}(z) = x$  zu berechnen
  - Oskar kann also den Klartext erhalten, ohne das Verschlüsselungsverfahren zu brechen

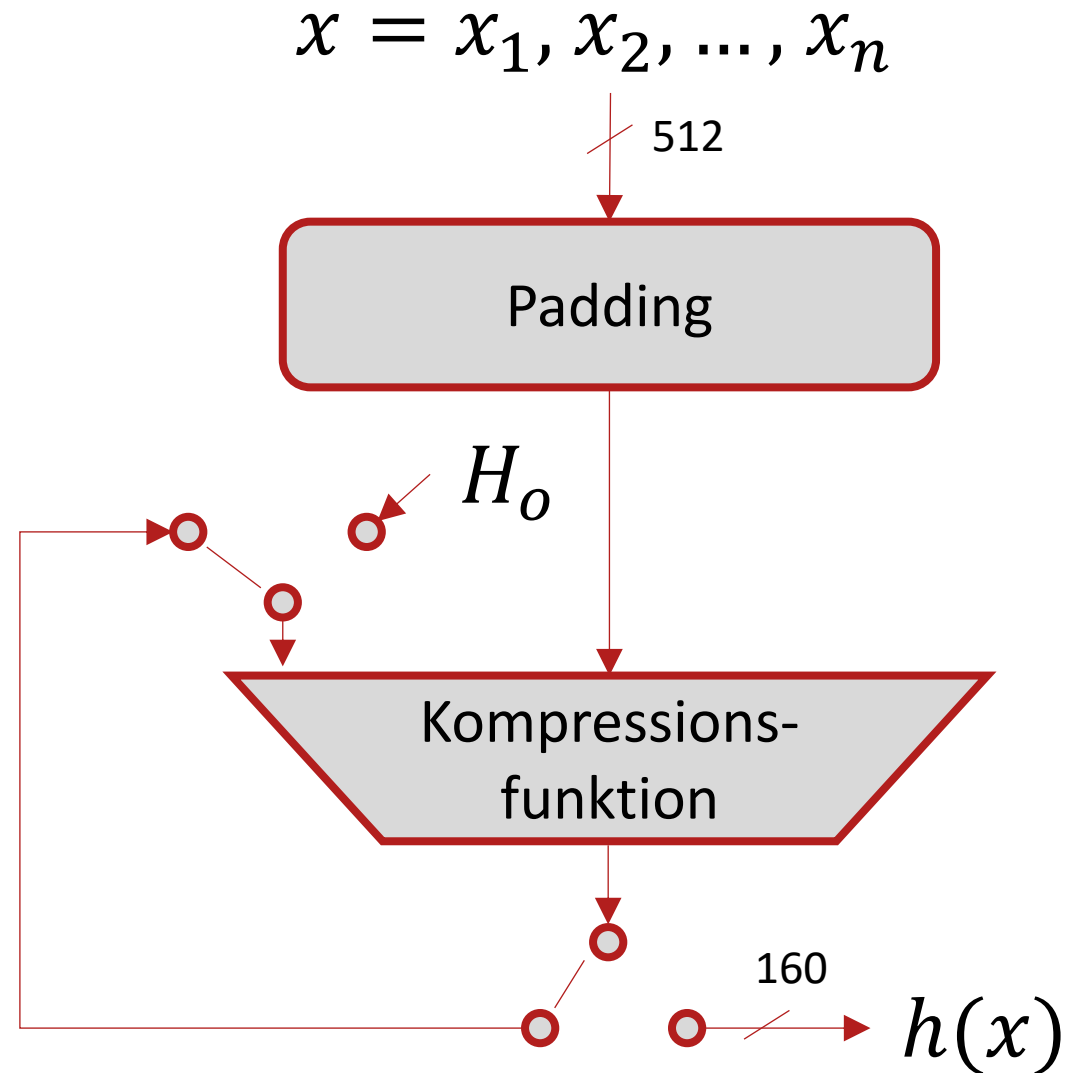
- Ziel:
  - Es soll unmöglich sein für eine gegebene Nachricht  $x_1$  eine Nachricht  $x_2$  zu finden, die auf den gleichen Hash-Wert abbildet ( $x_1 \neq x_2$ )
- Warum ist das wichtig:
  - Ein aktiver Angreifer könnte sonst Nachrichten austauschen
    - Original Nachricht:  $x_1, sig(h(x_1))$
    - Gefälschte Nachricht:  $x_2, sig(h(x_1))$
- Beachte:
  - Kollisionen müssen zwangsläufig vorkommen aufgrund der festen Ausgabelänge von Hash Funktionen
  - In der Praxis wählt man deswegen 128-512 Bit für den Ausgangswert, um solche Angriffe rechentechnisch unmöglich zu machen

- Ziel:
  - Es soll unmöglich sein ein Nachrichtenpaar  $x_1, x_2$  zu finden, die beide den gleichen Hash-Wert aufweisen ( $x_1 \neq x_2$ )
- Herausforderung:
  - Wie viele unterschiedl. Nachrichten braucht man, um Kollisionen zu finden?
    - Bei einem Ausgangswert von 128 Bit braucht man durchschnittlich  $2^{64}$  Nachrichten
  - Warum nicht  $2^{128} + 1$  Nachrichten?
    - Geburtstagparadoxon: Wie viele Gäste müssen anwesend sein, damit eine hohe Wahrscheinlichkeit besteht, dass 2 Gäste am selben Tag Geburtstag haben?
    - 50% Wahrscheinlichkeit bei 23 Personen, 90% Wahrscheinlichkeit bei 40 Personen
    - Bei Hash-Funktionen gibt es  $2^n$  Ausgangswerte (bei Geburtstagen nur 365)
    - Aus dem Geburtstagparadoxon lässt sich ableiten, dass  $\sqrt{2^n} = 2^{n/2}$  Nachrichten für eine Kollision benötigt werden



- Es gibt zwei Familien von Hash-Funktionen
  - MD4-Familie (wesentlich geläufiger in der Praxis)
  - Hash Funktionen auf Basis von Blockchiffren
- MD4-Familie (MD steht für Message Digest)
  - MD5, SHA-1, SHA-2 basieren alle auf MD4
  - Dabei wird die Verarbeitung auf 32-Bit Werten vorgenommen und alle Operationen sind boolesche Funktionen (AND, OR, XOR)
- SHA-3 gehört nicht direkt zur MD4-Familie weil es einen anderen internen Aufbau hat
- Für SHA-2 und SHA-3 gibt es die Ausgabelängen 224, 256, 384, 512 Bit

# Grober Aufbau von SHA-1 (MD4-Familie)



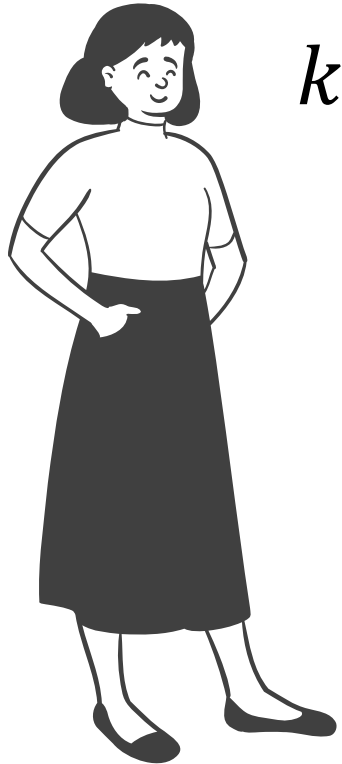
# Message Authentication Codes (MACs)

# Was sind MACs und warum nutzt man sie?

- Ein MAC ist eine kryptografische Prüfsumme
- Ähnlich wie digitale Signaturen gewährleisten MACs Nachrichten**integrität** und Nachrichten**authentisierung**
- Unterschied zu digitalen Signaturen
  - MACs basieren auf symmetrischen (statt asymmetrischen) Verfahren
  - MACs sind deutlich effizienter
  - Aber, MACs gewährleisten keine **Nichtzurückweisbarkeit**
  - Wie bei Hash-Funktionen kann die Nachrichtenlänge (Eingabe) beliebige Länge haben; die Prüfsumme (Ausgabe) hat eine feste Länge

# Prinzip von Message Authentication Codes (MACs)

Alice



$$k = (k_{AB})$$

$k_{pub}$

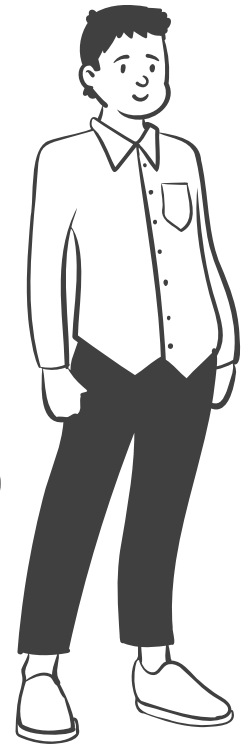


$(x, m)$



$$Verify_{k_{AB}}(m, x) = true/false$$

Bob



$$k = (k_{AB})$$

Select  $x$

$$m = MAC_{k_{AB}}(x)$$

- MACs können mit Blockchiffren realisiert werden; effizienter sind aber Hash-Funktionen
- Gene Tsudik beschreibt 1992 die Idee Hash Funktionen (damals MD4) für MACs zu nutzen
- Grundidee
  - Symmetrischer Schlüssel wird zusammen mit der Nachricht „gehashed“
- Naheliegende Ansätze für die Implementierung:
  - Secret-Prefix-MAC
    - $m = MAC_k(x) = h(k||x)$
  - Secret-Suffix-MAC
    - $m = MAC_k(x) = h(x||k)$

# Problem von Secret-Prefix-Mac

Alice

Oskar

Bob

$$X = x_1, x_2$$

$$m = h(k \| x_1, x_2)$$

$$\xleftarrow{x_1, m}$$

$$X_O = (x_1, x_2, x_3)$$

$$m_O = h(m, x_3)$$

$$\xleftarrow{x_O, m_O}$$

$$m' = h(k \| x_1, x_2, x_3)$$

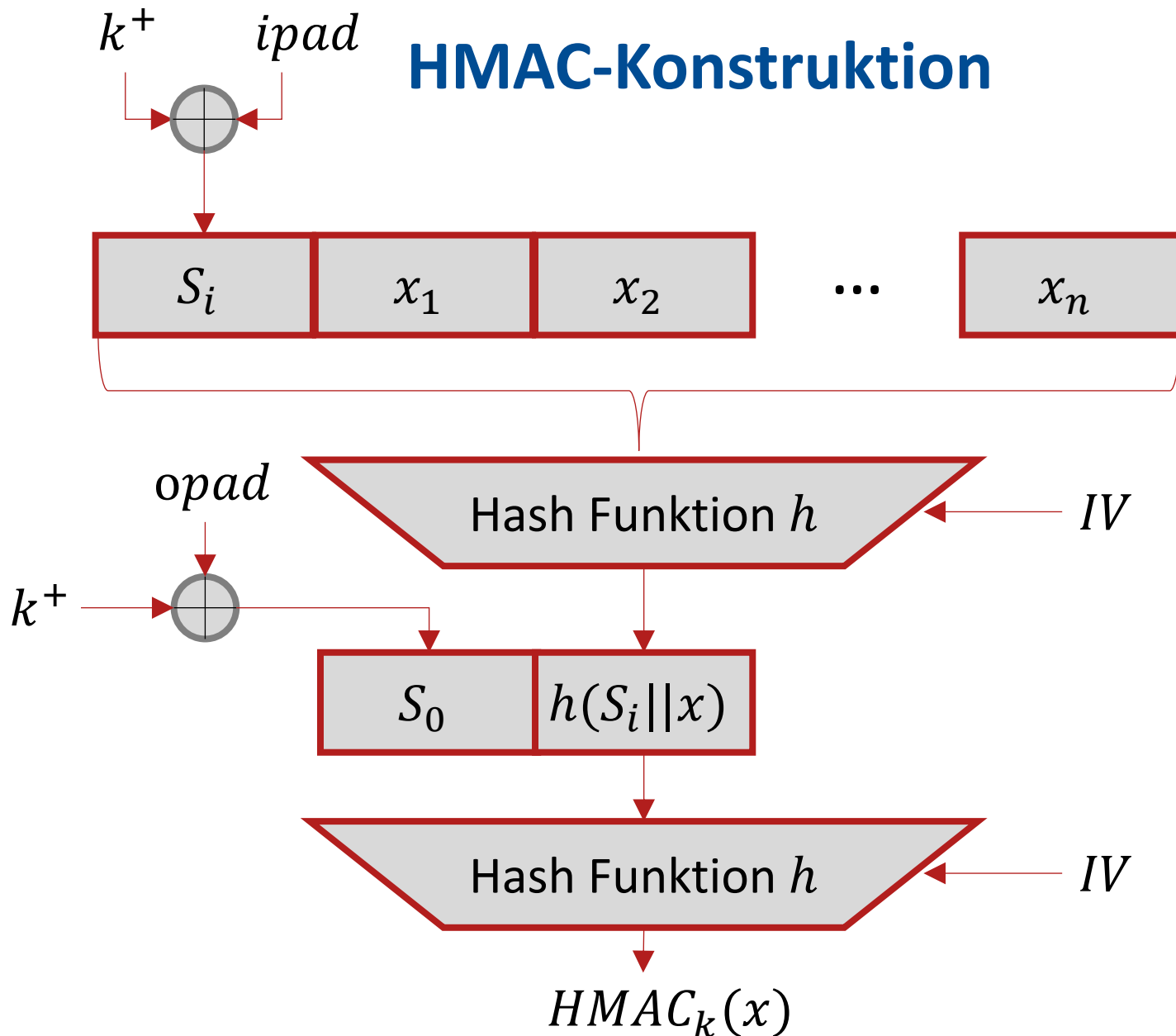
$$m' = m_O \quad \checkmark$$

- Oskar findet eine Kollision in der genutzten Hash Funktion:
$$h(x) = h(x_o)$$
- Wenn Bob nun die Nachricht  $x$  und die Prüfsumme  $m = h(x||k)$  auf dem Kanal verschickt, kann Oskar den Klartext durch  $x_o$  ersetzen
- Die Prüfsumme ist aufgrund der Kollision immer noch gültig, da
$$h(x||k) = h(x_o||k)$$
- Solche Angriffe sind insbesondere dann interessant wenn neue Angriffe auf Hash-Funktionen auftauchen



- 1996 von Mihir Bellare, Ran Canetti und Hugo Krawczyk vorgeschlagen
- Sicherheit des HMAC
  - Hohe Sicherheit
  - HMAC weist nicht die Schwachstellen von Secret-Prefix-MAC und Secret-Suffix-MAC auf
  - HMAC ist auch dann sicher wenn neue Angriffe auf die eingesetzte Hash-Funktion bekannt werden
- HMAC ist weit verbreitet und wird in TLS und IPSec genutzt

# HMAC-Konstruktion



## Schlüsselexpansion $k^+$

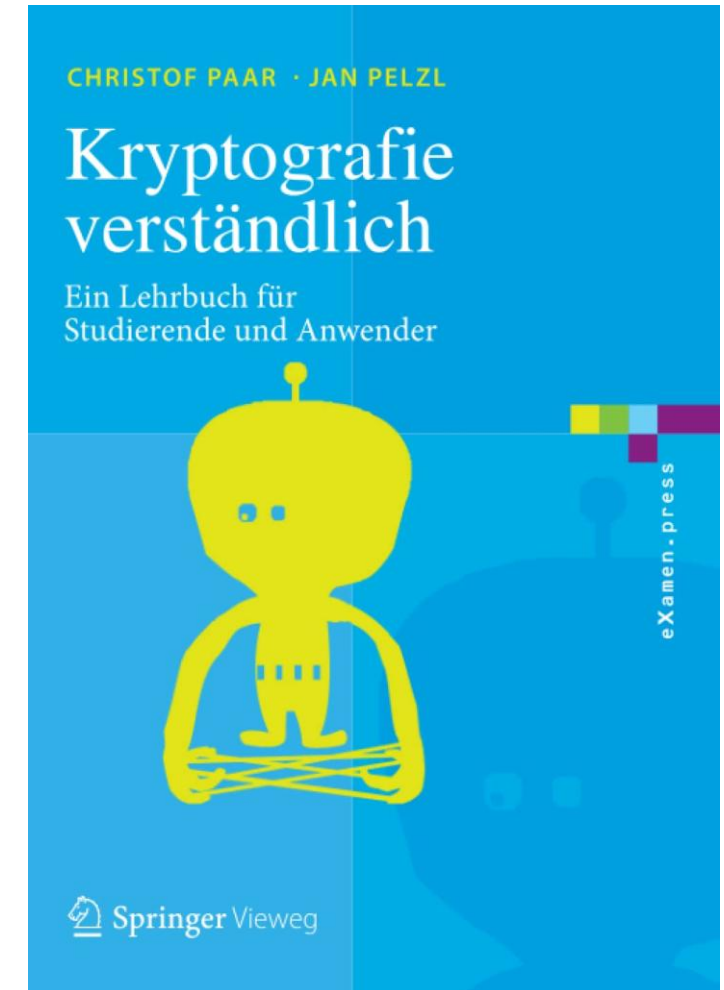
An den Schlüssel  $k$  werden so viele Nullen angehängt bis die Blockgröße (512 Bit bei SHA-1) der Hash-Funktion erreicht ist

## Masken ( $ipad, opad$ )

$ipad = 0011\ 0110, \dots, 0011\ 0110$  (0x36)  
 $opad = 0101\ 1100, \dots, 0101\ 1100$  (0x5C)

# Referenzen für Message Authentication Codes

- Die Vorlesungsfolien für dieses Thema sind zum großen Teil auf Basis des Buches „Kryptografie verständlich“ von C. Paar und J. Pelzl entstanden
- Es gibt ebenfalls eine dazugehörige Vorlesungsreihe „Einführung in die Kryptografie“ von C. Paar auf YouTube:  
<https://www.youtube.com/channel/UCuJu8DOJLMItMt8RcX1tdBw/videos>
- Teile des heutigen Vorlesungsinhalts basierend auf folgenden Veröffentlichungen:
  - G. Tsudik: Message Authentication with One-Way Hash Functions [\[Link\]](#) ACM SIGCOMM 1992
  - M. Bellare, R. Canetti, H. Krawczyk: Keying Hash Functions for Message Authentication [\[Link\]](#) CRYPTO 1996



Vorlesung Cybersicherheit, Sommersemester 2025

# Vielen Dank für Ihre Aufmerksamkeit

Prof. Dr.-Ing. Lucas Vincenzo Davi  
Fakultät für Informatik  
Arbeitsgruppe Systemsicherheit  
Universität Duisburg-Essen

© SYSSEC, Prof. Dr. Lucas Davi